Striver Strings - Basic and Easy Questions

1. Remove Outermost Paranthesis :

Problem Statement :

A valid parentheses string is defined by the following rules:

- It is the empty string "".
- If A is a valid parentheses string, then so is "(" + A + ")".
- If A and B are valid parentheses strings, then A + B is also valid.

A primitive valid parentheses string is a non-empty valid string that cannot be split into two or more non-empty valid parentheses strings.

Given a valid parentheses string s, your task is to remove the outermost parentheses from every primitive component of s and return the resulting string.

Examples :

Example 1:

```
Input: s = "((()))"
Output: "(())"
Explanation: The input string is a single primitive: "((()))".
Removing the outermost layer yields: "(())"
```

Example 2:

```
Input: s = "()(()())(())"
Output: "(()())()"
Explanation: Primitive decomposition: "()" + "(()())" + "(())"
After removing outermost parentheses: "" + "()()" + "()"
Final result: "(()())()".
```

Approach

- Initialize an empty result string to store the processed output.
- Initialize a counter (level) to track the depth of parentheses.
- Traverse through the string character by character.
- If the current character is '(', increment the level counter. If the level is greater than 1 (indicating we're inside a valid primitive), add '(' to the result string.
- If the current character is ')', decrement the level counter. If the level is greater than 0 (indicating we're still inside a valid primitive), add ')' to the result string.
- After the entire string has been traversed, return the result string.string

Python Code

In [1]:
```python
class Solution:
    # Function to remove outer parentheses
    def removeOuterParentheses(self, s):
        # Initialize result string
        result = ""
        # Initialize nesting level counter
        level = 0

        # Traverse the string
        for char in s:
            # If we encounter '(', increase the level
            if char == '(':
                # If we're inside a primitive, add '(' to result
                if level > 0:
                    result += char
                # Increase the nesting level for '('
                level += 1
            elif char == ')':
                # Decrease the nesting level for ')'
                level -= 1
                # If we're inside a primitive, add ')' to result
                if level > 0:
                    result += char

        # Return the final result after removing the outer parentheses
        return result
```

In [2]:
```python
# Example usage
s = "(()())(())"
sol = Solution()

# Get result
print(sol.removeOuterParentheses(s))
```

```
()()()
```

Complexity Analysis

Time Complexity:

```
The Time Complexity will be O(n), since we are performing a single traversal
of the string.
```

Space Complexity:

```
The Space Complexity will be O(1), since we are using a few variables to
track the current state.
```

2. Reverse Every Word in a String :

Problem Statement :

Given an input string, containing upper-case and lower-case letters, digits, and spaces(' '). A word is defined as a sequence of non-space characters. The words in s are separated by at least one space.

Return a string with the words in reverse order, concatenated by a single space.

Examples :

Example 1:

```
Input: s = "welcome to the jungle"
Output: "jungle the to welcome"
Explanation: The words in the input string are "welcome", "to", "the", and
"jungle". Reversing the order of these words gives "jungle", "the", "to",
and "welcome". The output string should have exactly one space between each
word.
```

Example 2:

```
Input: s = " amazing coding skills "
Output: "skills coding amazing"

Explanation: The input string has leading and trailing spaces, as well as
multiple spaces between the words "amazing", "coding", and "skills". After
trimming the leading and trailing spaces and reducing the multiple spaces
between words to a single space, the words are "amazing", "coding", and
"skills". Reversing the order of these words gives "skills", "coding", and
"amazing". The output string should not have any leading or trailing spaces
and should have exactly one space between each word.
```

Example 3:

```
Input: s = "openAI is innovative"
Output: "innovative is openAI"


Constraints:
1 <= s.length <= 104
s contains English letters (upper-case and lower-case), digits, and spaces '
'.
There is at least one word in s.
```

Brute Force Solution

Intuition

A brute force way to solve the problem will be to extract every word from the given string. Once all the words are extracted, the resulting string can be formed by adding all the words in the reverse order keeping a space in betweeen.

Approach

- First, the string is scanned to extract words, which are sequences of non-space characters.
- These words are stored in a list while ignoring extra spaces.
- Once all the words are collected, they are concatenated in reverse order to form the result.
- The words are joined with a single space between them, ensuring no trailing or leading spaces in the final output.

Python Code

In [1]:
```python
class Solution:
    # Function to reverse every word in the given string
    def reverseWords(self, s: str) -> str:
        string_length = len(s)  # Length of string

        # List to store the words present in string
        words = []

        # Pointers to mark the start and end of a word
        start = end = 0

        i = 0
        while i < string_length:

            # Finding the first character of a word (if any)
            while i < string_length and s[i] == ' ':
                i += 1

            # If no word is found, break
            if i >= string_length:
                break

            start = i  # Storing the index of first character of word

            # Finding the last character of the word
            while i < string_length and s[i] != ' ':
                i += 1

            end = i - 1  # Storing the index of last character of word

            # Add the found word to the list of words
            wordFound = s[start:end + 1]
            words.append(wordFound)

        ans = ""

        # Adding all the words to result in the reverse order
        for i in range(len(words) - 1, -1, -1):
            ans += words[i]

            # Adding spaces in between words
            if i != 0:
                ans += ' '

        return ans  # Return the stored result
```

In [2]:
```python
if __name__ == "__main__":
    s = " amazing coding skills "

    # Creating an instance of Solution class
    sol = Solution()

    # Function call to reverse every word in the given string
    ans = sol.reverseWords(s)

    # Output
    print("Input string:", s)
    print("After reversing every word:", ans)
```

```
Input string:  amazing coding skills
After reversing every word: skills coding amazing
```

Complexity Analysis

Time Complexity:

```
The Time Complexity will be O(n) where n is the length of the input string.

The input string is scanned once to extract words, taking O(n) time, where n
is the length of the input string. Each word is stored in a list and then
concatenated in reverse order, which also takes O(n).
```

Space Complexity:

```
The Space Complexity will be O(n).

The words list stores each extracted word, requiring O(k) space, where k is
the total number of characters in all words (essentially O(n)). The result
string requires O(n) space as well.
```

Optimal Solution

Intuition

The brute force approach used an additional list to store the words found in the string.

An optimal approach can be to reverse the words in-place. For this, the whole string is reversed in-place making each word's characters reversed. Now, each word is identified and its characters are reversed again to restore the correct character order within each word.

Extra spaces are moved to the end by shifting each word with the appropriate spacing.

Approach

- Reverse the entire input string first. Start iterating through the string to process each word.
- Words are identified as sequences of characters separated by spaces.
- For each word found, its characters are reversed individually while maintaining their relative positions in the sentence and moving them in front to trim down any unnecessary spaces.
- Any extra spaces at the end are removed before returning the final result.

Python Code

In [3]:
```python
class Solution:
    def reverseString(self, s, start, end):
        while start < end:
            s[start], s[end] = s[end], s[start]
            start += 1
            end -= 1

    def reverseWords(self, s: str) -> str:
        string_length = len(s)

        # Reverse the entire string
        s = list(s)
        self.reverseString(s, 0, string_length - 1)

        i = 0
        j = 0
        start = 0
        end = 0
```

```python
        while j < string_length:
            # Skip spaces
            while j < string_length and s[j] == ' ':
                j += 1
            if j == string_length:
                break

            start = i

            # Copy the word characters forward
            while j < string_length and s[j] != ' ':
                s[i] = s[j]
                i += 1
                j += 1

            end = i - 1

            # Reverse the current word using start and end
            self.reverseString(s, start, end)

            # Add a space after the word if it's not the last word
            if j < string_length:
                s[i] = ' '
                i += 1

        # Remove trailing space if present
        if i > 0 and s[i - 1] == ' ':
            i -= 1

        return "".join(s[:i])
```

In [4]:
```python
if __name__ == "__main__":
    s = " amazing coding skills "

    # Creating an instance of Solution class
    sol = Solution()

    # Function call to reverse every word in the given string
    ans = sol.reverseWords(s)

    # Output
    print("Input string:", s)
    print("After reversing every word:", ans)
```

```
Input string:  amazing coding skills
After reversing every word: skills coding amazing
```

Complexity Analysis

Time Complexity:

```
The Time Complexity will be O(n) where n is the length of the input string.

The input string is reversed firstly, taking O(n) time. The string is then
traversed taking another O(n) time.
Every word encountered in the string is again reversed taking overall O(n)
time.
```

Space Complexity:

```
The Space Complexity will be O(1).
```

```
There is no additional space used. The reversal is done in-place taking O(1)
or constant space.

Note that the space complexity for the Java solution will be O(N) because of
the conversion of the given String to a character array. This is done
because Strings are immutable in Java.
```

3. Largest Odd Number in a String

Problem Statement :

Given a string s, representing a large integer, the task is to return the largest-valued odd integer (as a string) that is a substring of the given string s.

The number returned should not have leading zero's. But the given input string may have leading zero. (If no odd number is found, then return empty string.)

Examples:

Example 1:

```
Input: s = "5347"
Output: "5347"
Explanation: The odd numbers formed by given strings are --> 5, 3, 53, 347,
5347. So the largest among all the possible odd numbers for given string
is 5347.
```

Example 2:

```
Input: s = "0214638"
Output: "21463"

Explanation: The different odd numbers that can be formed by the given
string are --> 1, 3, 21, 63, 463, 1463, 21463. We cannot include 021463 as
the number contains leading zero. So largest odd number in given string is
21463.
```ven string is 21463.
```

Intuition

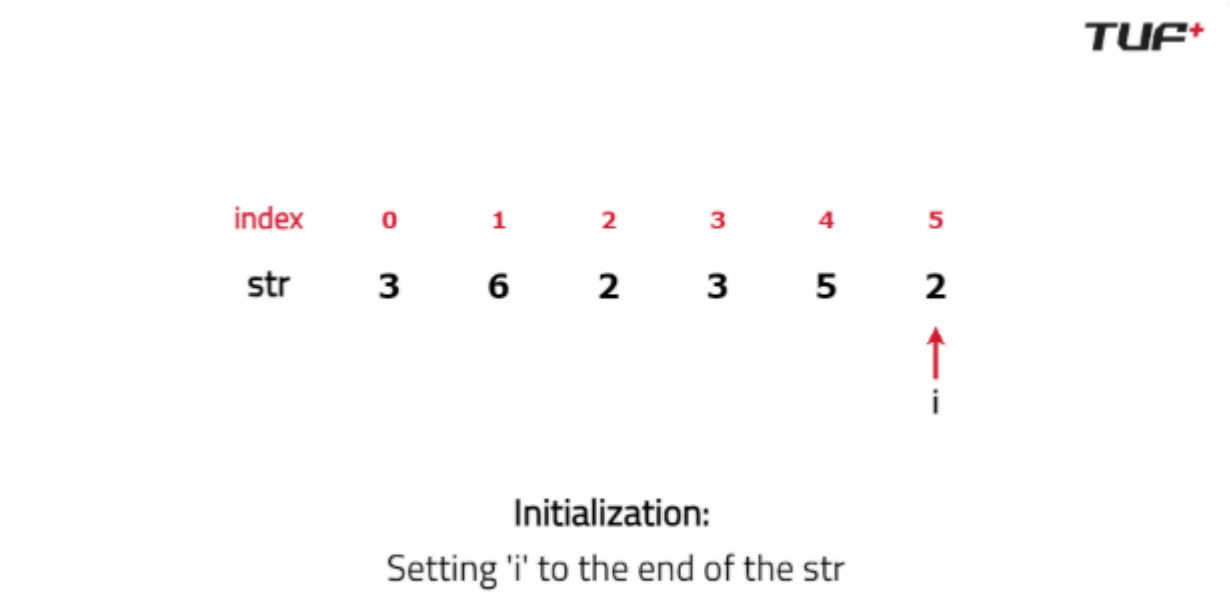
To determine the largest substring ending with an odd digit, start by iterating backward from the end of the number. This approach efficiently finds the rightmost odd digit by examining each character in reverse order. Once an odd digit is encountered, the substring from the beginning of the number up to and including this digit represents the largest possible odd-ending substring. This process leverages the fact that finding the last occurrence of an odd digit directly provides the longest valid substring.

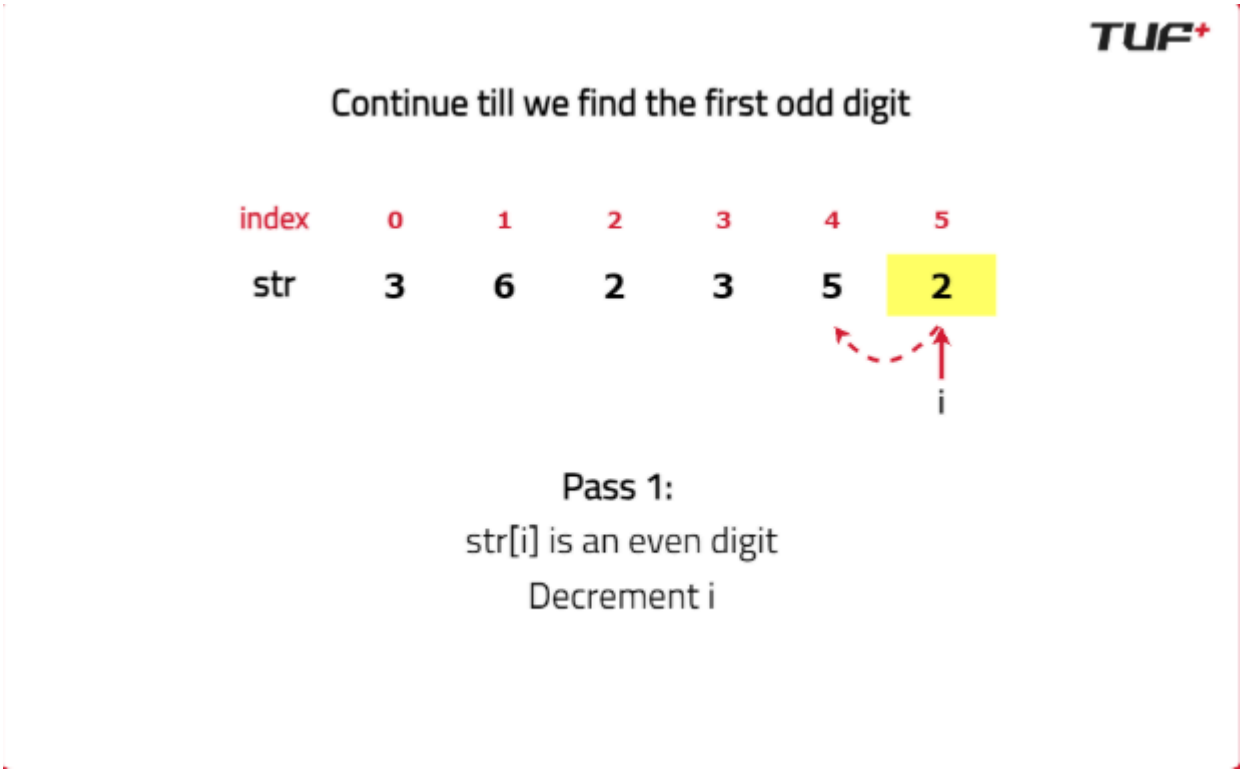
Approach

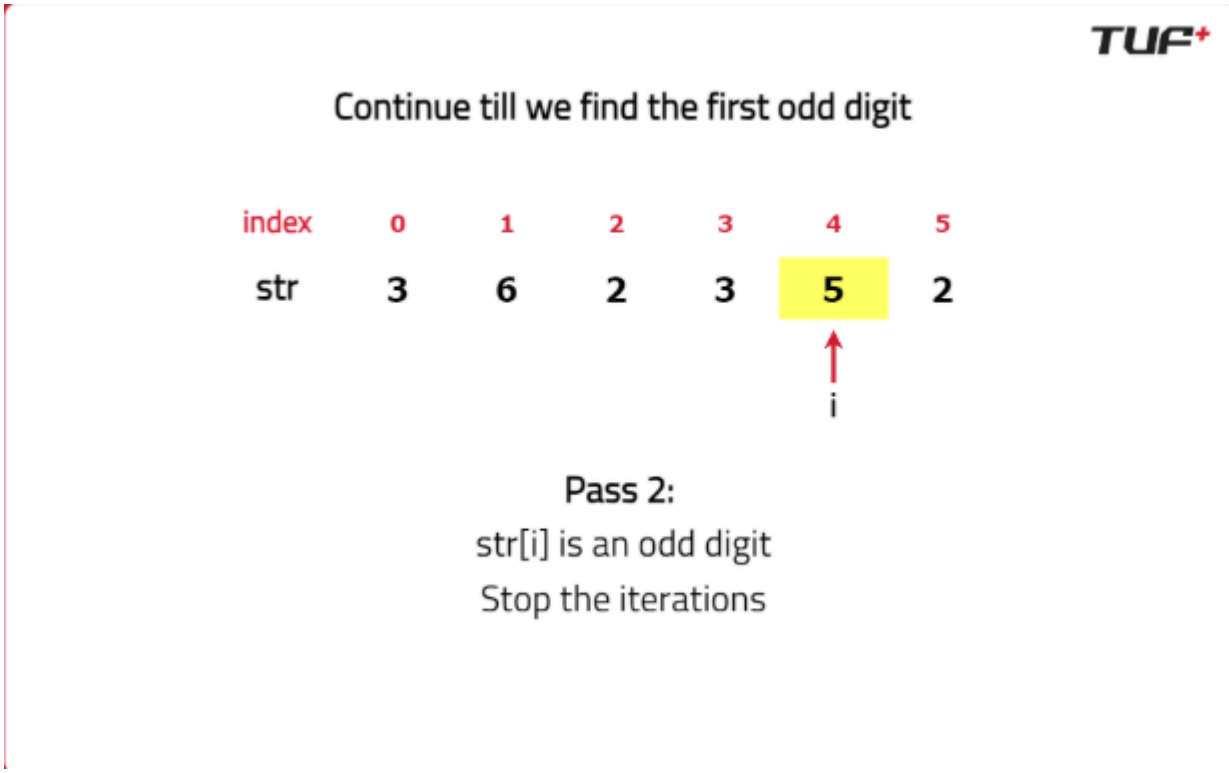
- Start by iterating through the string from the end towards the beginning to find the first odd digit. This digit marks the potential end of the largest odd number substring.
- Once an odd digit is found, skip any leading zeroes from the beginning of the string up to this odd digit.
```

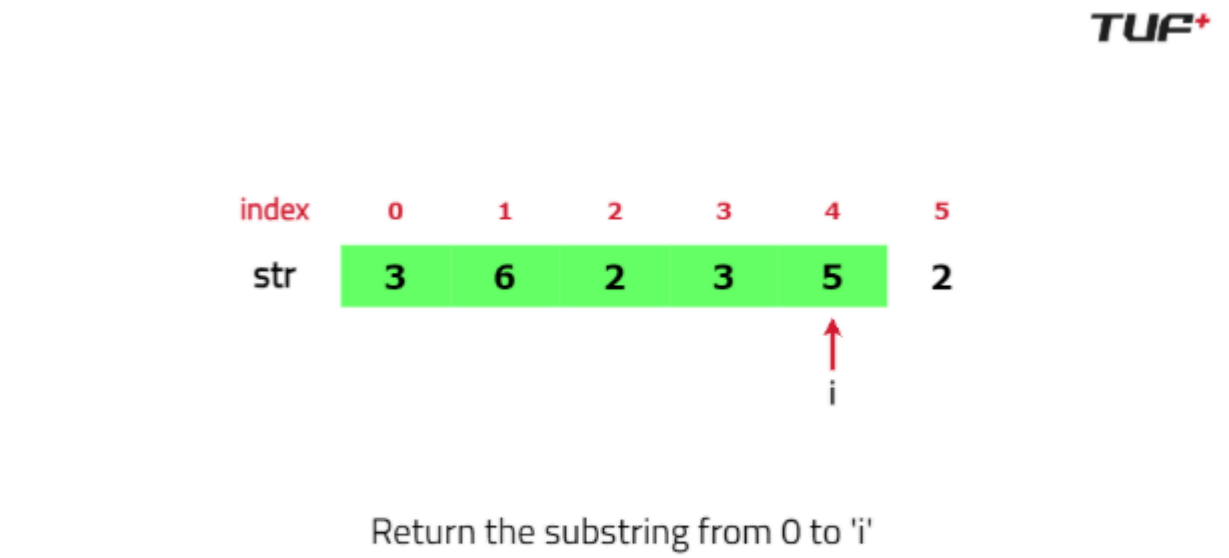
- Extract and return the substring starting after the leading zeroes and ending at the identified odd digit. This substring represents the largest odd integer without leading zeroes.

Dry Run









Python Code

In [5]:
```
class Solution:
    # Function to find the largest odd number
    # that is a substring of given string
    def largeOddNum(self, s: str) -> str:
        ind = -1

        # Iterate through the string from the end to beginning
        i = 0
        for i in range(len(s) - 1, -1, -1):
            # Break if an odd digit is found
            if (int(s[i]) % 2) == 1:
                ind = i
                break
```

```
        # Skipping any leading zeroes
        i = 0
        while i <= ind and s[i] == '0':
            i += 1

        # Return the largest odd number substring
        return s[i:ind + 1]

# Driver code
solution = Solution()
num = "504"
result = solution.largeOddNum(num)
print("Largest odd number:", result)
```

```
Largest odd number: 5
```

Complexity Analysis

Time Complexity:

```
The Time Complexity will be O(N), since the loop runs once through the
string of length N.
```

Space Complexity:

```
The Space Complexity will be O(N). The auxiliary space used is O(1) but if
the space for returned string is considered (which will be O(N) in the worst
case), the overall space complexity comes out to be O(N).
```

4. Longest Common Prefix

Problem Statement :

Given a string s, return the longest happy prefix of s. A happy prefix is a string that is both a proper prefix and a proper suffix. If no such prefix exists, return an empty string.

Example 1:

```
Input: s = "ababab"
Output: "abab"
Explanation: "abab" is the longest prefix which is also suffix. They can
overlap in the original string.
```

Example 2:

```
Input: s = "aaaa"
Output: "aaa"

Explanation: "aaa" is the longest prefix which is also a suffix in the
string "aaaa".
```

Example 3:

```
Input: s = "abc"
Output: ""
```

```
Explanation: "aaa" is the longest prefix which is also a suffix in the
string "aaaa".
```

Constraints:

```
1 <= s.length <= 104.
```

Intuition

The goal is to find the longest prefix of the string s that is also a proper suffix. This means:

- The prefix and suffix must match character by character.
- It must be proper, so it cannot be the entire string itself.

This is a classic case of pattern overlap, and it matches perfectly with the idea of the **LPS (Longest Prefix Suffix) array used in the KMP (Knuth-Morris-Pratt) string matching algorithm.** ing.tring.

Approach

- Compute the Longest Prefix Suffix (LPS) array using the KMP preprocessing technique.
- The LPS array stores the length of the longest proper prefix which is also a suffix for each prefix of the string.
- The final value in the LPS array indicates the length of the longest happy prefix.
- Extract the substring from the start of the string up to this length and return it.
- If the length is zero, return an empty string.

Python Code

In [1]:
```python
class Solution:
    # Compute the Longest prefix suffix array for the combined string
    def computeLPS(self, s):
        n = len(s)  # size of string

        # To store the longest prefix suffix
        LPS = [0] * n

        i, j = 1, 0

        # Iterate on the string
        while i < n:
            # If the character matches
            if s[i] == s[j]:
                LPS[i] = j + 1
                i += 1
                j += 1

            # Otherwise
            else:
                # Trace back j pointer till it does not match
                while j > 0 and s[i] != s[j]:
                    j = LPS[j - 1]

                # If a match is found
                if s[i] == s[j]:
                    LPS[i] = j + 1
                    j += 1
                i += 1
```

```python
        return LPS  # Return the computed LPS array

    '''
    Function to find the longest happy
    prefix of the given string
    '''
    def lps(self, s):
        LPS = self.computeLPS(s)
        return s[:LPS[-1]]
```

In [2]:
```python
if __name__ == "__main__":
    s = "ababab"

    # Creating an instance of the solution class
    sol = Solution()

    '''
    Function call to find the longest
    happy prefix of the given string
    '''
    ans = sol.lps(s)

    print("The longest happy prefix of the given string is:", ans)
```

```
The longest happy prefix of the given string is: abab
```

Complexity Analysis

Time Complexity:

```
The Time Complexity will be O(N), where N is the length of given string.
Building the LPS array takes O(N) time. Extracting the substring also takes
O(k) time, where k is the length of the longest happy prefix (and k ≤ n), so
it's O(N) in the worst case. Thus, the overall time complexity is O(N) +
O(N) which is O(N).
```

Space Complexity:

```
The Space Complexity will be O(N), because an additional array LPS of size n
is used to store the prefix-suffix lengths.
```

5. Isomorphic String

Problem Statement :

Given two strings s and t, determine if they are isomorphic. Two strings s and t are isomorphic if the characters in s can be replaced to get t.

All occurrences of a character must be replaced with another character while preserving the order of characters. No two characters may map to the same character, but a character may map to itself.

Examples :

Example 1:

```
Input: s = "egg" , t = "add"
Output: true
```

```
Explanation: The 'e' in string s can be replaced with 'a' of string t. The
'g' in string s can be replaced with 'd' of t. Hence all characters in s can
be replaced to get t.
```

Example 2:

```
Input: s = "apple" , t = "bbnbm"
Output: false

Explanation: It can be proved that no solution exists for this example. All
the "b"s in t have to take places of "a", "p", "l", which requires "p" to be
mapped to "b", but that makes it impossible for "p" at index 2 (0-indexed)
to become "n". Thus no solution exists.
```

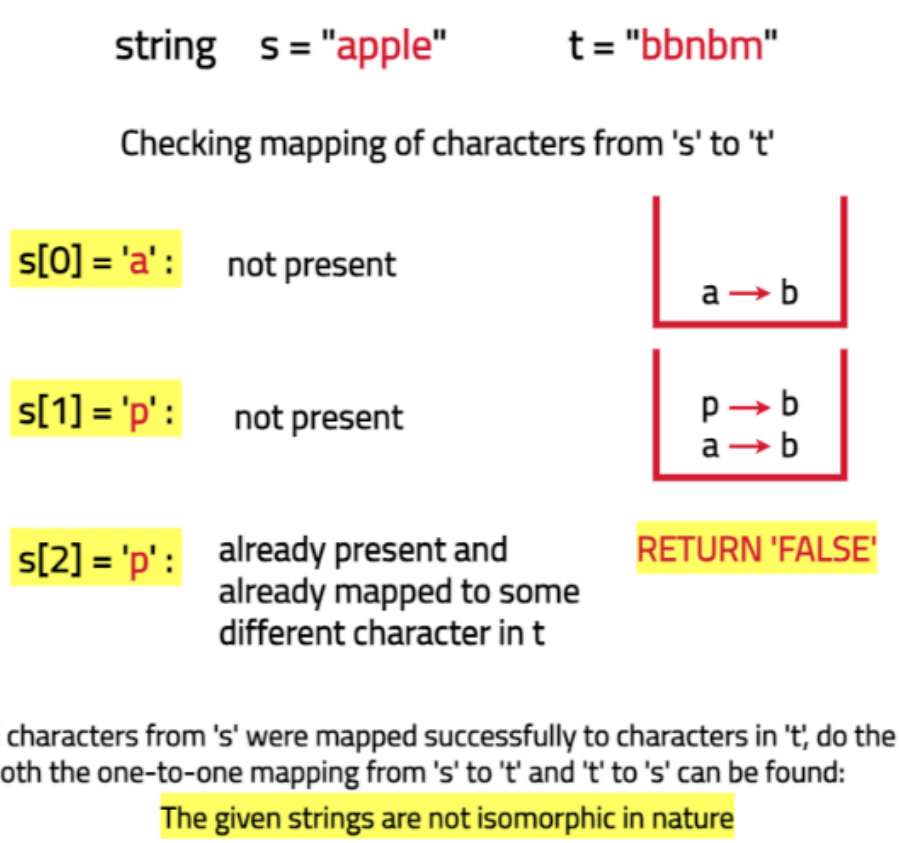
Intuition

To determine if two strings are isomorphic, the key insight is to recognize that two strings are isomorphic if the characters in one string can be replaced to get the other string. This can be efficiently checked by ensuring that there is a consistent mapping of characters from the first string to the second string and vice versa. The challenge lies in maintaining this mapping as the strings are traversed, ensuring that each character in the first string maps uniquely to a character in the second string, and that no two characters in the first string map to the same character in the second string.ly.

Approach

- Initialize two arrays of size 256 (to cover all ASCII characters) to store the last seen positions of characters in both strings. This helps in tracking the mapping between characters.
- Iterate through each character in the strings simultaneously. For each character, compare the last seen positions stored in the arrays. If the positions do not match, it indicates an inconsistent mapping, and the strings are not isomorphic.
- If the positions match, update the arrays with the current index (incremented by 1 to avoid the default value of 0). This ensures that the mapping is consistent throughout the strings.
- After completing the iteration, if no inconsistencies in the mappings are found, the strings are confirmed to be isomorphic. If any inconsistency is found during the iteration, return false immediately.

Dry Run



Python Code

In [3]:
```
class Solution:
    def isomorphicString(self, s, t):
        # Arrays to store the last seen positions of characters in s and t
        m1, m2 = [0] * 256, [0] * 256

        # Length of the string
        n = len(s)

        # Iterate through each character in the strings
        for i in range(n):
            # If the last seen positions of the current characters don't match, return false
            if m1[ord(s[i])] != m2[ord(t[i])]:
                return False

            # Update the last seen positions
            m1[ord(s[i])] = i + 1
            m2[ord(t[i])] = i + 1

        # If all characters match, return true
        return True
```

In [4]:
```
# Main method for testing
if __name__ == "__main__":
    solution = Solution()
    s = "egg"
    t = "add"
    if solution.isomorphicString(s, t):
        print("Strings are isomorphic.")
    else:
        print("Strings are not isomorphic.")
```

```
Strings are isomorphic.
```

Complexity Analysis

Time Complexity:

```
The Time Complexity will be O(N) where N is the length of the input strings,
due to the single loop iterating through each character.
```

Space Complexity:

```
The Space Complexity will be O(K), since the space used by the arrays is
constant (256 fixed size) irrespective of input size.
```

6. Check whether one String is a Rotation of Another String

Problem Statement :

Given two strings s and goal, return true if and only if s can become goal after some number of shifts on s. A shift on s consists of moving the leftmost character of s to the rightmost position.

For example, if s = "abcde", then it will be "bcdea" after one shift.

Examples :

Example 1:

```
Input : s = "abcde" , goal = "cdeab"
Output: true

Explanation :

After performing 2 shifts we can achieve the goal string from string s.

After first shift the string s is => bcdea
After second shift the string s is => cdeab.
```

Example 2:

```
Input: s = "abcde", goal = "adeac"
Output: false

Explanation:

Any number of shift operations cannot convert string s to string goal.
```

Brute Force Solution

Intuition

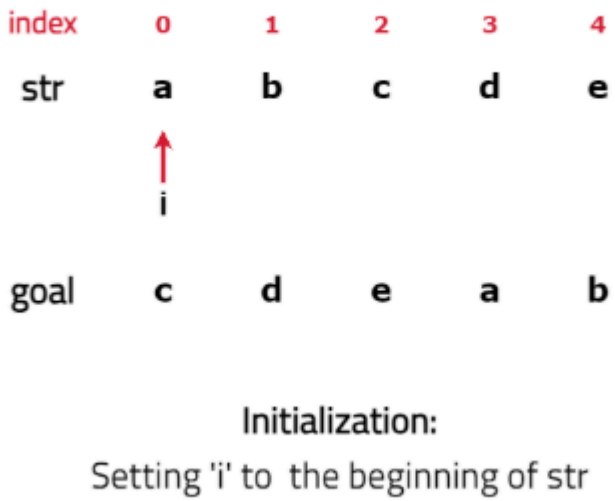
To address this problem, a solution could be to generate all possible rotations of the string and verify if any of these rotations match the target `goal` string. The approach involves systematically creating each

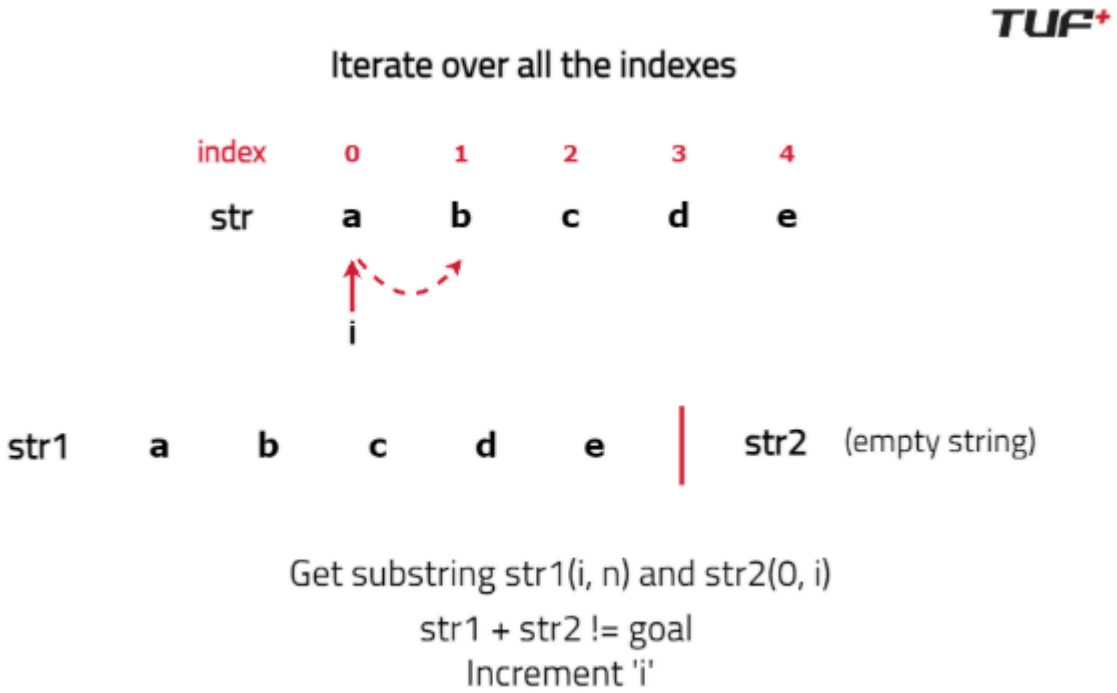
rotation and comparing it against the `goal`, which ensures that if a matching rotation exists, it will be identified.

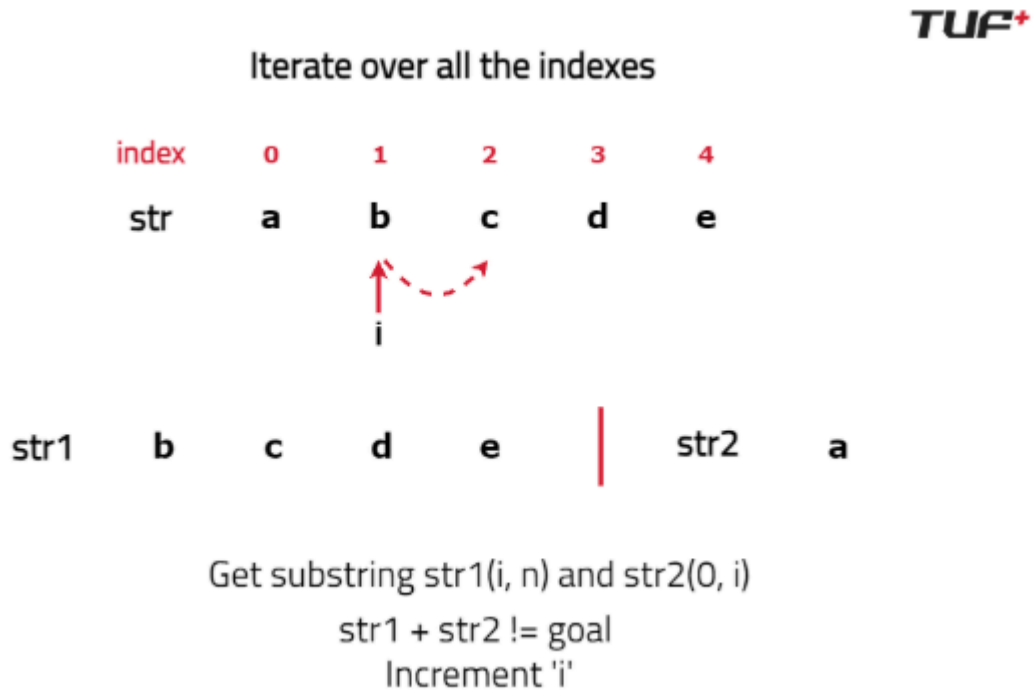
Approach

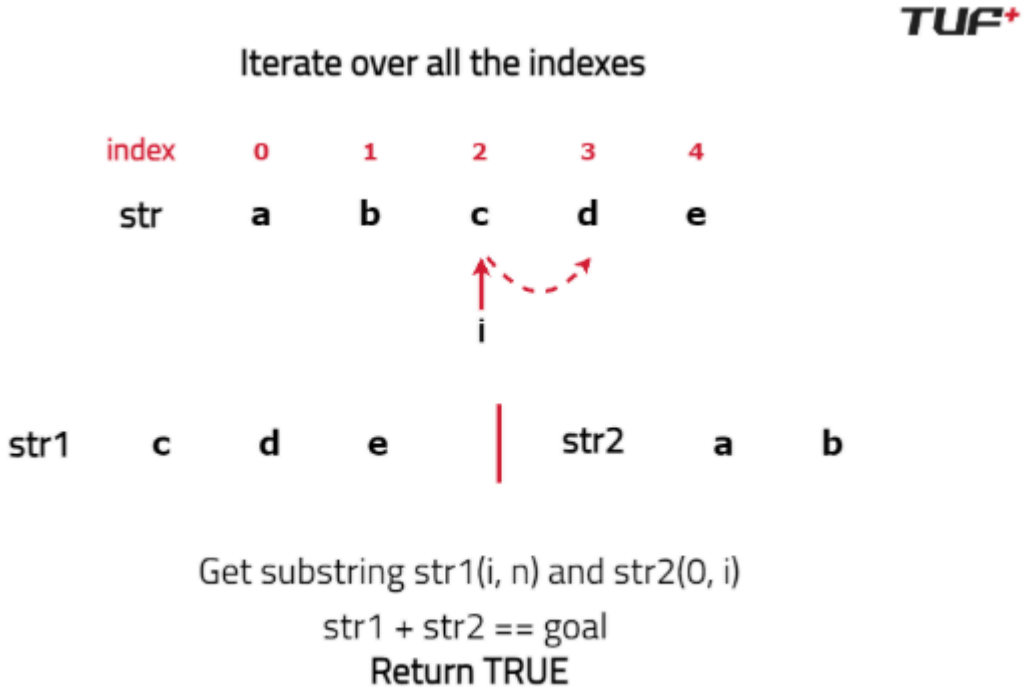
- First generate all possible rotations of the string by rearranging its character using the substring method.
- For each rotation created, check if it is the same as the goal string.
- If any rotation matches the goal, return true; otherwise, after testing all rotations, return false.

Dry Run









Python Code

In [1]:
```
class Solution:
    def rotateString(self, s: str, goal: str) -> bool:
   # Strings must be same length to be rotations of each other
        if len(s) != len(goal):
            return False
        # Try all possible rotations of s
        for i in range(len(s)):
            rotated = s[i:] + s[:i]  # Create a new rotation
            if rotated == goal:
                return True
        return False

# Test cases
sol = Solution()
print(sol.rotateString("abcde", "cdeab"))  # Output: True
print(sol.rotateString("abcde", "abced"))  # Output: False
```

```
True
False
```

Complexity Analysis

Time Complexity:

```
The Time Complexity will be O(N^2), since we are generating N rotations and
each comparison takes O(N) time.
```

Space Complexity:

```
The Space Complexity will be O(N) for the space needed to store each rotated
string.
```

Optimal Solution

Intuition

The optimal approach solves the problem by concatenating s with itself to form s + s. This way, all possible rotations of s are included as substrings in s + s, so we just need to check if goal is a substring of s + s.alse.

Approach

- Create a new string by concatenating s with itself, resulting in s + s.
- Check if goal is a substring of s + s.
- If goal is found within s + s, return true; otherwise, return false.

Dry Run

Python Code

In [2]:
```
class Solution:
# Strings must be of the same length to be rotations of each other
    def rotateString(self, s: str, goal: str) -> bool:
        if len(s) != len(goal):
            return False
        doubled_s = s + s  # Concatenate s with itself
        return goal in doubled_s  # Check if goal is a substring of s + s

# Test cases
sol = Solution()
print(sol.rotateString("abcde", "cdeab"))
print(sol.rotateString("abcde", "abced"))
```

```
True
False
```

Complexity Analysis

Time Complexity:

```
The Time Complexity will be O(N), because checking for a substring in s + s
is linear in time.
```

Space Complexity:

```
The Space Complexity will be O(N) for the space needed to store the
concatenated string s + s.
```

7. Check if 2 Strings are Anagrams of each other

Problem Statement :

Given two strings s and t, return true if t is an anagram of s, and false otherwise.

An Anagram is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all the original letters exactly once.

Examples :

Example 1:

```
Input: s = "anagram" , t = "nagaram"
Output: true

Explanation: We can rearrange the characters of string s to get string t as
frequency of all characters from both strings is same.
```

Example 2:

```
Input: s = "dog" , t = "cat"
Output: false
```

```
Explanation: We cannot rearrange the characters of string s to get string t
as frequency of all characters from both strings is not same.
```

Brute Force Solution

Intuition

The letters of an anagram should form identical sequences if alphabetically sorted. By furthering this thought process a method to check for anagrams would be sorting both strings. By sorting both strings and then comparing them, we can easily determine if they contain the same characters in the same frequencies.

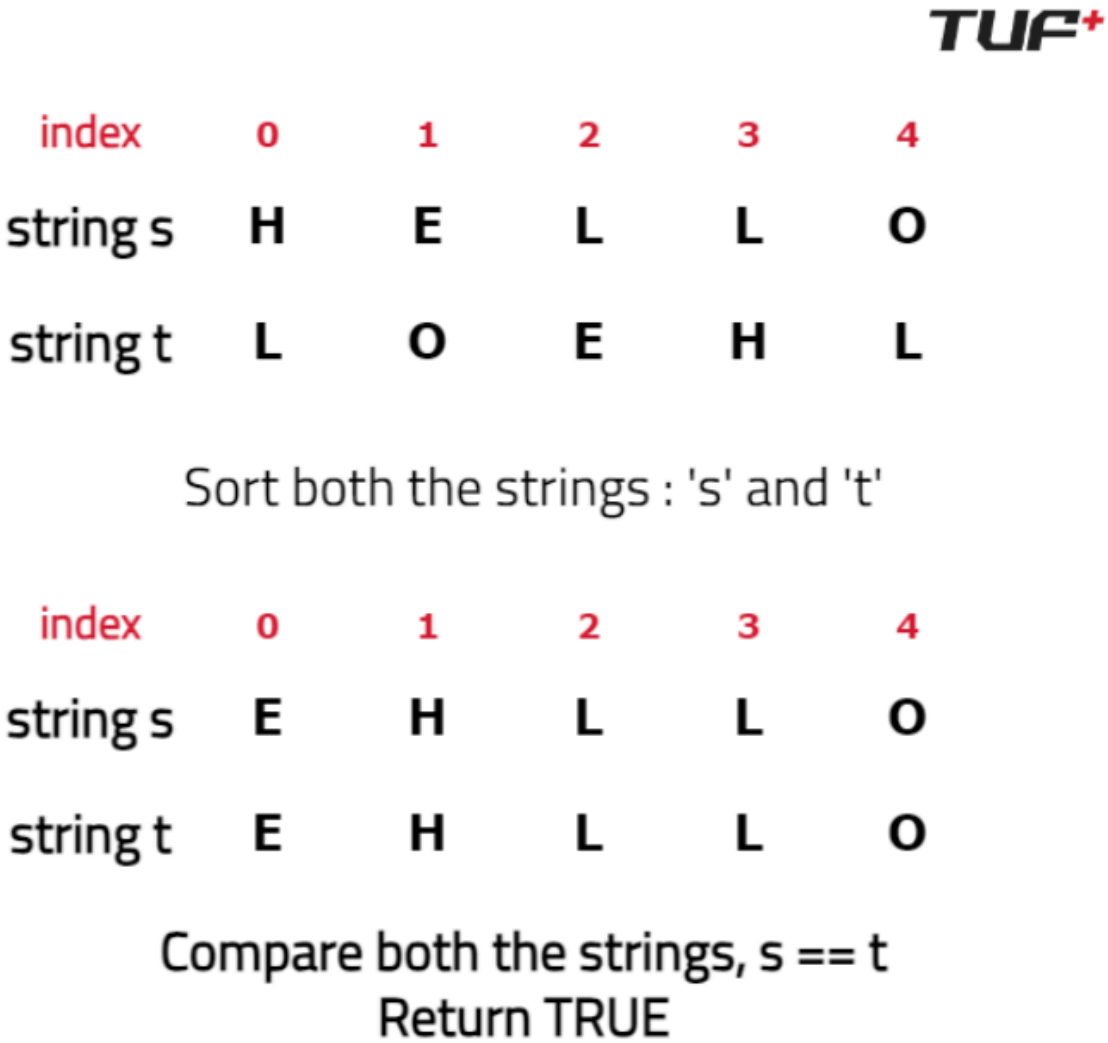
Approach

- Sort the characters of both strings using an inbuilt sort function, so that if they are anagrams, the sorted strings will be identical.
-

Compare the sorted versions of both strings. If they match, the original strings are anagrams; otherwise, they are no

- Return true if the sorted strings are identical, otherwise return false.lse.

Dry Run



Python Code

In [3]:
```python
class Solution:
    def anagramStrings(self, s, t):
        # If lengths are not equal, they cannot be anagrams
        if len(s) != len(t):
            return False

        # Sort both strings and compare
        return sorted(s) == sorted(t)

if __name__ == "__main__":
    solution = Solution()
    str1 = "INTEGER"
    str2 = "TEGERNI"
    result = solution.anagramStrings(str1, str2)
    print("True" if result else "False")
```

```
True
```

Complexity Analysis

Time Complexity:

```
The Time Complexity will be O(N log N) because we are sorting each string.
```

Space Complexity:

```
The Space Complexity will be O(1) as no additional data structures are used.
Note that for Java, the space complexity will be O(N) due to the creation of
additional character arrays. And for Python, the space complexity will be
O(N) due to the use of sorted() function, which creates a new string to hold
the sorted string.
```

Optimal Solution

Intuition

An alternative approach to checking anagrams is to check the number of times each character in both strings appears through frequency counting. The frequency counting method for checking if two strings are anagrams is based on the idea that two strings are anagrams if they have the same characters appearing the same number of times. By counting the frequency of each character in the first string and then decreasing the count for each character in the second string, we can ensure that both strings have identical character distributions. If any character count does not return to zero, the strings are not anagrams.

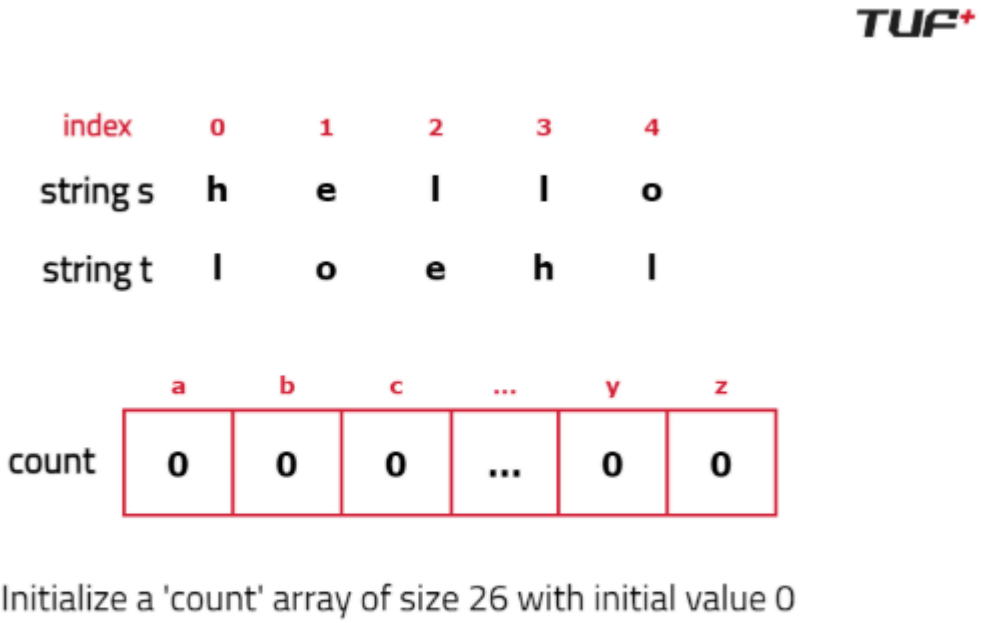
Approach

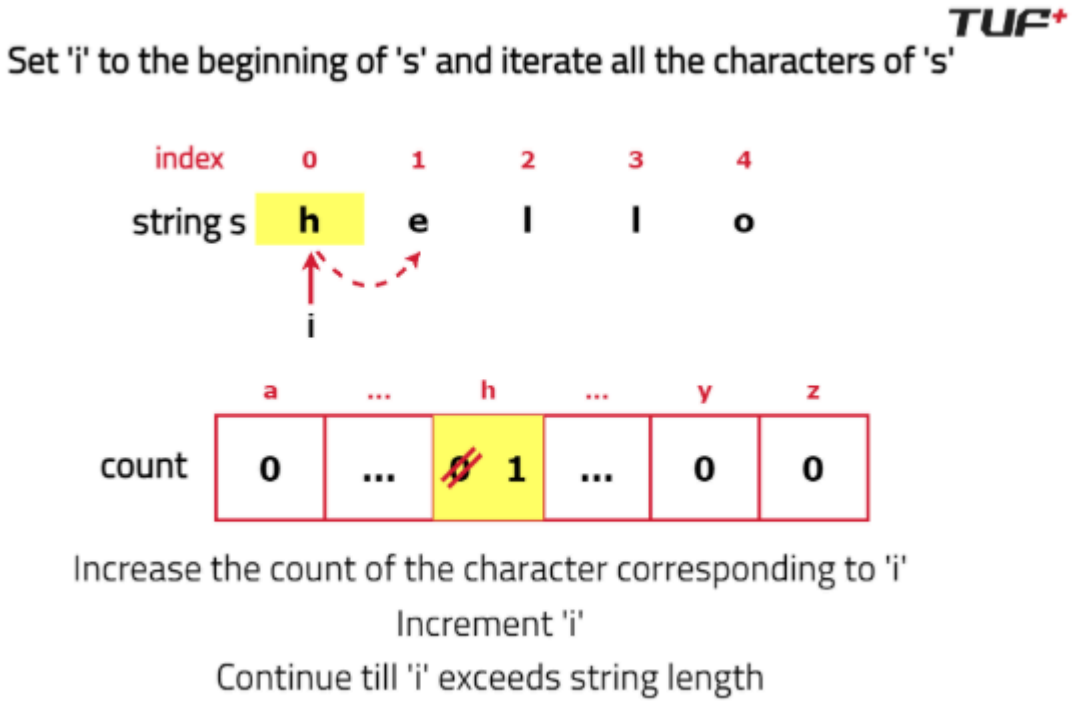
- Initialize a frequency array of size 26 to count the occurrences of each letter in the first string (Str1). Each index of the array represents a letter from 'a' to 'z'.
-

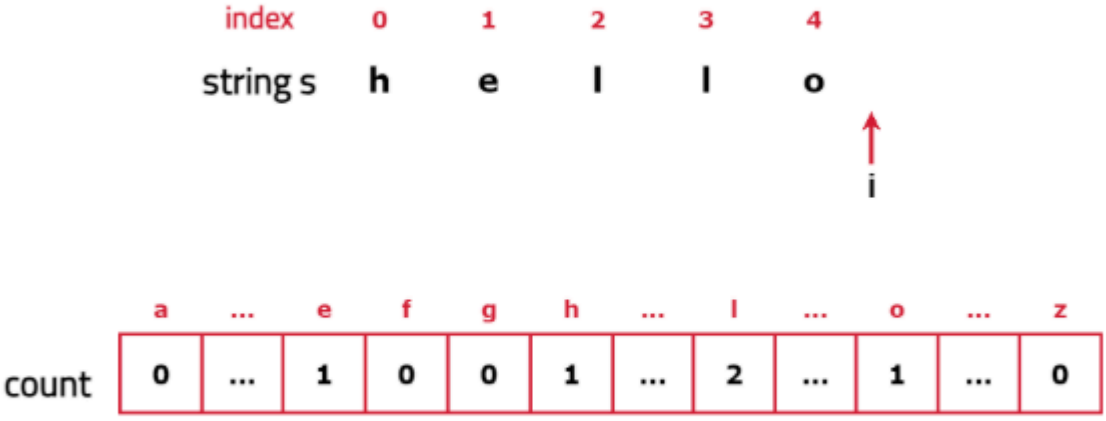
Iterate through the second string (Str2) and decrease the count in the frequency array for each letter found in Str2. This ensures we are balancing out the counts from Str

- Check the frequency array. If all counts return to zero, both strings have identical character frequencies and are anagrams. If any count is not zero, the strings are not anagrams.ams.

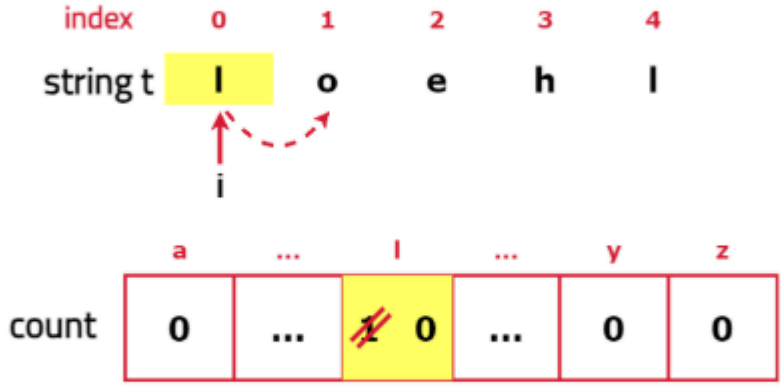
Dry Run







Set 'i' to the beginning of 't' and iterate all the characters of 't'



Decrease the count of the character corresponding to 'i'

Increment 'i'

Continue till 'i' exceeds string length

index	0	1	2	3	4
string t	l	o	e	h	l

i

	a	...	e	f	g	h	...	l	...	o	...	z
count	0	...	0	0	0	0	...	0	...	0	...	0

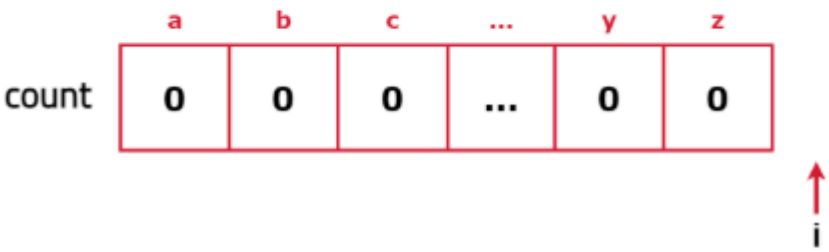
Set 'i' to the beginning of 'count' and iterate all the elements of 'count'

	a	b	c	...	y	z
count	0	0	0	...	0	0

i

count[i] == 0, Increment 'i'

Continue till 'i' exceeds array size



'i' has exceeded array size, so we stop the iterations
All the elements of count are equal to 0, so return TRUE

Python Code

In [4]:
```
class Solution:
    def anagramStrings(self, s, t):
        # Edge Cases
        if len(s) != len(t):
            return False

        # To store the count of each character
        count = [0] * 26

        # Count occurrence of each character in first string
        for c in s:
            count[ord(c) - ord('a')] += 1

        # Decrement the count for each character in the second string
        for c in t:
            count[ord(c) - ord('a')] -= 1

        # Check for count of every character
        for i in count:
            # If the count is not zero
            if i != 0:
                return False # Return false

        # Otherwise strings are anagram
        return True
```

The ord() function in Python returns the Unicode code point (an integer) for a given character. Syntax : ord(character).

Here's what happens:

- `ord('h')` = **104**
- `ord('a')` = **97**
- Difference = **104 - 97 = 7**

This gives you the **zero-based alphabetical index** of `'h'` relative to `'a'`.

So:

- `'a'` → 0
- `'b'` → 1
- ...
- `'h'` → 7

If you want **1-based order** (where `'a'` = 1), just add 1:

```python
position = ord('h') - ord('a') + 1  # 8

```

✅ `'h'` is the **8th letter** in the alphabet.

In [6]:
```python
if __name__ == "__main__":
    str1 = "integer"
    str2 = "tegerni"

    # Creating an instance of Solution class
    sol = Solution()

    '''Function call to find out
    whether two strings are anagram'''
    result = sol.anagramStrings(str1, str2)

    # Output
    if result:
        print("The given strings are anagrams.")
    else:
        print("The given strings are not anagrams.")
```

```
The given strings are anagrams.
```

Complexity Analysis

Time Complexity:

```
The Time Complexity will be O(N), where N is the length of the string
```

Space Complexity:

```
The Space Complexity will be O(1), as there is always a constant-size array
(of length 26) used to store the frequencies that does not depend on the
```

length of the strings.